

SP-4 — Multi-Provider Redesign (Trackers screen removal, generic Credentials, parallel search, sync)

Status: specification only — implementation in phases that follow.

Filed: 2026-05-12 per operator directives during the v1.2.16 release cycle.

Problem statement (operator's words, paraphrased)

1. **The Trackers screen exposes a "current tracker" picker, which is wrong.** The product **MUST** work with ALL enabled+configured providers simultaneously. The Trackers screen as a switch-between-providers surface is deprecated.
2. **API performs search + sync across ALL enabled providers in parallel, emits results as async events to subscribers.** Search results UI shows results from every responding provider with per-row labels identifying which provider returned which row.
3. **Per-provider configuration screen reached by tapping the provider name in the Menu.** That screen **MUST** support:
 - assigning one of the user's stored Credentials
 - creating new Credentials inline
 - adding mirror URLs
 - listing all pre-installed mirrors
 - all other per-provider options
 - connection probing per mirror
 - sync ON / OFF toggle for the provider

4. **CLONE existing provider** — duplicate a provider with a new display name + new URL set, so users can search multiple instances of the same provider type (e.g. two rutracker mirrors on different hosts, including LAN-hosted Tracker instances on the local network).
5. **All provider parameters sync via Lava API.** Sync settings of the app gate which sub-sets sync and at what cadence.
6. **Generic Credentials** as their own entity:
 - User creates a Credentials entry by name (e.g. "My main creds.")
 - Each Credentials has a type: username+password, token, api-key, or any of the supported authentication shapes.
 - Each Credentials carries the actual secret values for its type.
 - **One Credentials entry can be assigned to multiple providers simultaneously** (e.g. "My main creds." used for both RuTracker and RuTor).
 - Credentials sync via the Lava API and are end-to-end encrypted on both client and API sides. No leak of any kind permitted.
7. **Removed entries MUST also be removed from backups.** Once a user removes anything, a re-sync is mandatory.
8. **Full test coverage** — unit + integration + Compose UI Challenges. Anti-bluff per §6.J: every test asserts on user-visible state, falsifiability rehearsal per test, no @Ignore without tracking issue, no verify-only assertions.

Why this is multi-phase

The scope crosses every layer of the app + the Go API + the data model:

- **Domain model:** introduce CredentialsEntry, ProviderConfig, SyncToggle types. Migrate existing ProviderCredentialManager (Phase-11 per-provider username+password) to the new generic shape.
- **Persistence:** Room schema changes (new tables for credentials, provider config, sync toggle). Migration from the v17 schema. Backup-exclusion rules audit (settings.xml is already excluded as of v1.2.16; new credentials table needs encrypted-at-rest semantics AND sync-back semantics).
- **Network:** new Go API endpoints for credentials + provider config sync, with end-to-end encryption (operator-supplied key material;

the existing per-build LAVA_AUTH crypto chain is the starting point but credentials need a separate per-user secret derivation).

- **SDK:** LavaTrackerSdk learns to fan-out search/browse/auth across N providers in parallel and emit per-provider events on a SharedFlow. Backpressure + cancellation semantics specified.
- **UI:**
 - Search results screen: render per-provider rows with provider labels + provider color dot.
 - Menu: tap on provider row → per-provider config screen (new).
 - Per-provider config screen: assign / create / pick credentials, list + add mirrors, probe, clone-with-new-name, sync toggle.
 - Trackers screen: remove the picker entirely. The screen either goes away or becomes a per-provider summary (TBD in phase plan).
- **Tests:** every layer needs unit + integration + Compose UI Challenge coverage. §6.J primary-on-user-visible-state. Anti-bluff rehearsals recorded per commit.
- **Documentation:** user guide, developer guide, architecture diagram updates. The Continuation document tracks phase status.

A realistic phased breakdown (see docs/superpowers/plans/ for the implementation plan that follows this design):

- **Phase A — Generic Credentials model.** Domain types + Room migration + per-build encrypted-at-rest persistence + unit tests + Challenge for the create-credentials flow.
- **Phase B — Per-provider config screen.** Compose screen reachable from Menu tap on provider row. Assign existing credentials, add mirror URL, probe. Per-provider sync ON/OFF toggle. Challenge for full screen interactions.
- **Phase C — Trackers screen removal.** Replace the screen with a multi-provider summary OR delete from the navigation graph. C04-C08 Challenges remapped or rewritten honestly.
- **Phase D — Multi-provider parallel search SDK.** SharedFlow-based fan-out in LavaTrackerSdk. Per-provider result labeling. Search results UI updated to render labels. Cancellation + backpressure tests.
- **Phase E — Credentials sync via Lava API.** New Go endpoints /v1/credentials (PUT/DELETE/GET) with per-user key encryption. Client-side sync mechanism + retry. End-to-end encryption tests.
- **Phase F — Provider clone-with-new-name.** UI clone affordance. Domain support for "multiple instances of the same provider type with different mirror sets". Challenge for clone+search the clone.

- **Phase G — Removal-syncs-to-backup.** Soft-delete with sync-up semantics so a removal propagates to other devices and survives reinstall via backup. Backup-restore test on a second emulator.
- **Phase H — Documentation refresh.** User guide, developer guide, diagrams, CONTINUATION.md final state.

Each phase ships its own commit (or commit chain) and its own distribute round. Anti-bluff is enforced phase-by-phase: a phase that cannot produce a green Challenge Test against the real production stack is not marked complete.

Out of scope for SP-4

- **iOS client** — Lava is Android-only as of v1.2.16.
- **Cross-tracker deduplication** (treating two mirrors of the same rutracker post as one logical search result) — separate SP if the operator requests it later.
- **Per-credential MFA** — the credential types in scope are username+password, token, api-key. Adding TOTP / push-MFA is a future SP.

Operator's invariants reasserted in the design

- §6.J anti-bluff: every Challenge Test asserts on user-visible state. No verify-only. No @Ignore without an issue. No mock-the-SUT.
- §6.H credential security: credentials never appear in tracked files, in logs, or in non-encrypted persistence. All credential paths through LavaAuth style codegen-pepper-derived AES-GCM keys for at-rest; the per-user-key-derivation for sync is a Phase E design item.
- §6.P distribution versioning: each phase that ships an artifact bumps versionCode strictly + carries a CHANGELOG entry + per-version snapshot.
- §6.S CONTINUATION.md: every phase that lands updates CONTINUATION.md in the same commit.
- §6.W mirrors: GitHub + GitLab only, both updated in lockstep.

Phase A + B detailed design (locked 2026-05-12)

After brainstorming with the operator on 2026-05-12 (post-v1.2.16-distribute), the following decisions are locked for the first implementation cycle. Phases C-H remain at the high-level scoping in the section above and will get their own detailed-design appendices when their cycles begin.

Operator decisions during brainstorming

1. **Sequencing:** A2 — Phase A complete + Phase B FULL (mirrors + creds assignment + sync toggle + clone + anonymous mode) in this cycle.
2. **Credential types in scope:** Username+Password, API Key, Bearer Token, Cookie/Session string. All four supported in v1.
3. **Encryption:** AES-256-GCM with a per-user passphrase. Key derived via PBKDF2-SHA-256 (200_000 iterations). Zero-knowledge: API stores only encrypted blobs.
4. **Passphrase UX:** Lazy + in-process cache. Prompt only when user opens Credentials for the first time; derived key kept in memory for the lifetime of the process; re-prompt after force-stop.

Phase A — Generic Credentials data model

Module: `:core:credentials` (extends the existing module, which today houses `ProviderCredentialManager` + `CredentialEncryptor` + `ProviderConfigRepository` + `CredentialsRepository`). The existing classes stay in place; new ones are added alongside, and the legacy `ProviderCredentials` shape is migrated to the new `CredentialsEntry` shape via the Room migration documented below.

Domain model (in `:core:credentials:api`):

```
sealed interface CredentialSecret {
    data class UsernamePassword(val username: String, val password: String) : CredentialSecret
    data class ApiKey(val key: String) : CredentialSecret
    data class BearerToken(val token: String) : CredentialSecret
    data class CookieSession(val cookie: String) : CredentialSecret
}

enum class CredentialType { USERNAME_PASSWORD, API_KEY, BEARER_TOKEN, COOKIE_SESSION }
```

```

data class CredentialsEntry(
    val id: String,           // UUID v4
    val displayName: String,  // user-given, e.g. "My main
                             creds."
    val type:
        CredentialType,      // discriminator for the sealed shape
    val secret: CredentialSecret, // decrypted; never persisted
                             unencrypted
    val createdAtUtc: Long,
    val updatedAtUtc: Long,
)

interface CredentialsRepository {
    suspend fun list(): List<CredentialsEntry>
    suspend fun observe(): Flow<List<CredentialsEntry>>
    suspend fun get(id: String): CredentialsEntry?
    suspend fun upsert(entry: CredentialsEntry)
    suspend fun delete(id: String)
}

interface ProviderCredentialBinding {
    suspend fun bind(providerId: String, credentialId: String)
    suspend fun unbind(providerId: String)
    suspend fun observeBinding(providerId: String): Flow<String?>
}

```

Storage (Room, new schema bump from v8 to v9):

- Table `credentials_entry`: columns `id` (TEXT PK), `display_name` (TEXT NOT NULL), `type` (TEXT NOT NULL), `ciphertext` (BLOB NOT NULL — 12-byte AES-GCM nonce + ciphertext + 16-byte tag concatenated), `created_at` (INTEGER NOT NULL), `updated_at` (INTEGER NOT NULL).
- Table `provider_credential_binding`: `provider_id` (TEXT PK), `credential_id` (TEXT NOT NULL, FK → `credentials_entry.id` ON DELETE SET NULL).
- Migration 8 → 9: creates both tables. Existing `ProviderCredentialManager` rows migrated to one `CredentialsEntry` per row (display name = <providerId> default) + matching binding; legacy table is dropped after migration verifies row count.

Encryption:

- File: `core/credentials/impl/src/main/kotlin/lava/credentials/impl/crypto/CredentialsCrypto.kt`.
- AES-256-GCM via `javax.crypto.Cipher`. Per-entry random 12-byte nonce. 16-byte authentication tag (standard GCM).

- Key derivation: PBKDF2 with HMAC-SHA-256, 200_000 iterations, 32-byte random salt persisted in `settings.xml` under a fixed key `credentials_kdf_salt_v1` (non-secret).
- Verifier: HMAC-SHA-256(derivedKey, "lava-credentials-v1") persisted alongside the salt under `credentials_verifier_v1`. Wrong passphrase → verifier mismatch → reject without decryption attempt (avoids ciphertext oracle).
- Passphrase cache: in-memory `AtomicReference<DerivedKey>` on the `:core:credentials:impl` singleton scope; cleared on `Application.onLowMemory()` and on force-stop (Android kills the process → cache gone).

ViewModel & screen for credentials list / create / edit:

- New `:feature:credentials_manager` module. Orbit MVI.
- `CredentialsManagerScreen` reachable from Menu's existing "Provider Credentials" entry (line ~204 of `MenuScreen.kt` — "Provider Credentials", the entry already exists but routes to the legacy per-provider screen; rewire it to the new screen).
- Sub-flow: passphrase unlock dialog → list of credentials → tap to edit / "Add new" FAB → create-credentials sheet with type picker + per-type form fields.

Tests (Phase A):

- `CredentialsCryptoTest` (JVM): round-trip; wrong passphrase rejected via verifier; tampered ciphertext rejected via GCM tag.
- `CredentialsRepositoryImplTest` (JVM, real Room in-memory): upsert
 - observe + delete; provider-binding CRUD; legacy-migration test uses a v17 SQLite snapshot + asserts the migrated rows have the expected display names.
- `Challenge27CredentialsCreateAndAssignTest` (Compose UI on emulator): full create + assign flow.
- `Challenge31PassphraseUnlockFlowTest` (Compose UI): wrong-then-right unlock flow asserts on user-visible error + decrypted list.

Phase B — Per-provider configuration screen

New module: `:feature:provider_config` (Compose, Orbit MVI, applies `lava.android.feature` plugin).

Navigation:

- Menu provider Surface becomes clickable (the existing surface already has an `onClick` for sign-out — change it to route to `openProviderConfig(providerId)`; sign-out moves to a trailing icon with confirmation dialog).
- New extension function `addProviderConfig` registered in `:core:navigation`.

Screen structure (single Composable, `Column(verticalScroll(...))` — no nested `LazyColumn` per §6.Q):

1. **Header:** provider color dot + `displayName` + `descriptor.trackerId`
 - current connection status icon.
2. **Sync section:** `SwitchRow("Sync this provider", state.syncEnabled)`. New `provider_sync_toggle(provider_id TEXT PK, enabled INTEGER NOT NULL)` table. Toggling writes to that table + queues a sync-outbox row for Phase E uploader.
3. **Credentials section:** shows currently-bound `CredentialsEntry.displayName` or "None — anonymous". Two buttons:
 - "Assign existing..." → opens `CredentialsPickerSheet` (`ModalBottomSheet`) listing all entries; tap one → bind via `ProviderCredentialBinding.bind(providerId, credentialId)`.
 - "Create new..." → opens the `CredentialsCreateSheet` from Phase A (reused); on save the new entry is auto-bound to the current provider.
4. **Mirrors section:** for every entry in `descriptor.baseUrls` (pre-installed) + every `UserMirror(provider_id, url)` from `UserMirrorRepository`, render `MirrorRow(host, status, isPrimary, isUserAdded)`. Per row: probe icon → `ProbeMirrorUseCase` (HTTP HEAD with the existing Ktor client). Add new mirror: text field at the bottom + button.
5. **Anonymous toggle:** shown only when `descriptor.supportsAnonymous` is true. Switch toggles a new pref `provider_anonymous_${providerId}`.
6. **Clone provider:** button at bottom. Opens dialog "Clone {name} as..." with two fields: new display name + new primary mirror URL. Confirm creates a row in `new cloned_providers(synthetic_id TEXT PK, source_tracker_id TEXT, display_name TEXT, primary_url TEXT)` table. `LavaTrackerSdk.listAvailableTrackers()` is extended to union the cloned rows as `TrackerDescriptor` overlays.

Sync wiring (honest-stub for this cycle):

- A new `:core:sync` module is introduced with a `SyncOutbox` interface + Room-backed impl. Operations on `provider_sync_toggle`, `provider_credential_binding`, `cloned_providers`, and `user_mirror` enqueue an outbox row.
- The Phase-E uploader (separate cycle) will drain the outbox; this cycle just queues. CHANGELOG documents this as honest-stub.

Tests (Phase B):

- `ProviderSyncToggleRepositoryImplTest (JVM)`: toggle persists; outbox row created.
- `CloneProviderUseCaseTest (JVM)`: clone produces synthetic descriptor visible through `listAvailableTrackers()`.
- `ProbeMirrorUseCaseTest (JVM, MockWebServer)`: 200, 4xx, timeout, TLS-failure.
- `Challenge28PerProviderSyncToggleTest (Compose UI on emulator)`: toggle sync OFF → reopen screen → still OFF; outbox table contains entry.
- `Challenge29AddCustomMirrorTest`: add `rutracker.foo` mirror → row appears in list.
- `Challenge30CloneProviderTest`: clone "RuTracker.org" as "RuTracker EU" with URL `https://rutracker.eu` → returns to Menu → both rows visible.

Implementation sequencing (subagent-driven per operator approval)

Per the operator's "you can begin using Subagents-Driven approach" hint, the work decomposes into independent subagent tasks per phase:

- **Subagent S-A1**: Phase A foundation — `:core:credentials` module scaffold + Room migration + crypto + repository. Pure Kotlin; no UI.
- **Subagent S-A2**: Phase A UI — `:feature:credentials_manager` module + passphrase unlock + create/edit/delete UI + Menu rewire.
- **Subagent S-B1**: Phase B screen scaffolding — `:feature:provider_config` module + navigation + screen skeleton with header + sync section.
- **Subagent S-B2**: Phase B credentials assignment + clone + anonymous + sync outbox wiring.
- **Subagent S-B3**: Phase B mirrors section + probe + add-mirror flow.

- **Subagent S-T1:** Test author — write all Challenge Tests + their KDoc-recorded falsifiability rehearsals, run on the live emulator, record evidence under `.lava-ci-evidence/sp4-phase-ab/`.

Each subagent's brief explicitly carries the §6.J + §6.L + §6.S mandate. Main agent reviews each subagent's output against this design spec before integration.

CHANGELOG + distribute plan

- Version bump: 1.2.16-1036 → 1.2.17-1037 / 2.3.5-2305 → 2.3.6-2306.
- CHANGELOG entry covering Phase A + Phase B feature additions, with honest disclosure that:
 - Sync outbox queues but does not yet upload (Phase E owed).
 - Mirrors probe UI exists; per-mirror auto-failover is Phase D.
- Per-version snapshot, pepper rotation, last-version update — same §6.P pattern as v1.2.16.
- Both APKs distributed via Firebase to operator tester group.
- Go API local restart at v2.3.6-2306; kept booted.

Forensic anchor

Operator's directives during the v1.2.15 / v1.2.16 release cycle (2026-05-12 sequence after the C03 + Cloudflare + anti-bluff cycle):

"When opening the Trackers screen, we can switch between the Providers!? This is not how we have requested in last iterations to be! WE MUST work with all enabled and configured providers!"

"The whole Trackers screen is not needed anymore! We MUST make sure we can open Provider configuration screen my tap on it from Menu on its name."

"Make sure we can CLONE existing provider and assign it a new name and new URL so practically users can sync and search multiple instances running on various hosting platforms of particular Tracker (Provider) type we support!"

"Make sure all Providers DO SYNC and can be toggled for syncing ON and OFF!"

"The Credentials we create MUST BE generic! ... heavily protected and encrypted on both sides ... user can choose for one or more providers the Credentials. Like this 'My main creds.' can be used at the same time for RuTracker provider and RuTor provider!"

The 19th §6.L invocation (same 2026-05-12 wave) reaffirms the anti-bluff mandate: every phase MUST guarantee real-user-visible functionality, not green-tests-with-broken-features.